



RESUMO DA MISSÃO: LOG-01 & 02

Tema: O Despertar do Código e o Poder das Variáveis

Instrutor: Romisson Oliveira



1. ONDE O CÓDIGO ACONTECE?

Hoje não usamos editores complexos. Fomos direto na fonte: o **Console do Navegador**. O Javascript inicialmente foi criado para rodar comandos nos navegadores e cada navegador possui um motor que interpreta seus comandos. No passado, isto vinha se tornando um problema pois não havia uma padronização nos comandos em Javascript. Foi para isto que foi criado o padrão **ECMAScript**. Sendo uma tentativa (bem sucedida) de padronizar os comandos e como os navegadores iriam interpretar a linguagem em suas aplicações. Isto possibilitou um avanço significativo na tecnologia.

Atualmente, podemos rodar Javascript também em outros locais, como nossa máquina e em servidores através do NodeJS - que é um motor de interpretação *run-time* baseado no V8 do Google Chrome para rodar Javascript fora do ambiente do navegador (client-side).

Porém tudo isto é apenas uma curiosidade para que você fique por dentro do assunto. Como estamos estudando aplicações **front-end**, isto é, aquelas que rodam no lado do cliente (navegadores), hoje, ao invés de abrir o VSCode, abrimos a ferramenta *DevTools* (F12) para rodar nossos primeiros comando Javascript a partir do navegador.

- **O Motor:** O Google Chrome (V8) possui um motor que interpreta JavaScript em tempo real.
 - **Ferramenta:** Em uma página web aberta no navegador, faça:
 - Botão Direito > Inspeccionar > Aba Console.
 - **Por que?** Porque para entender a lógica, primeiro precisamos sentir o "pulso" da linguagem sem as facilidades de um editor. E foi isso que fizemos hoje.
-

2. SAÍDA DE DADOS (COMUNICAÇÃO)

Aprendemos as duas formas principais de fazer o computador "falar":

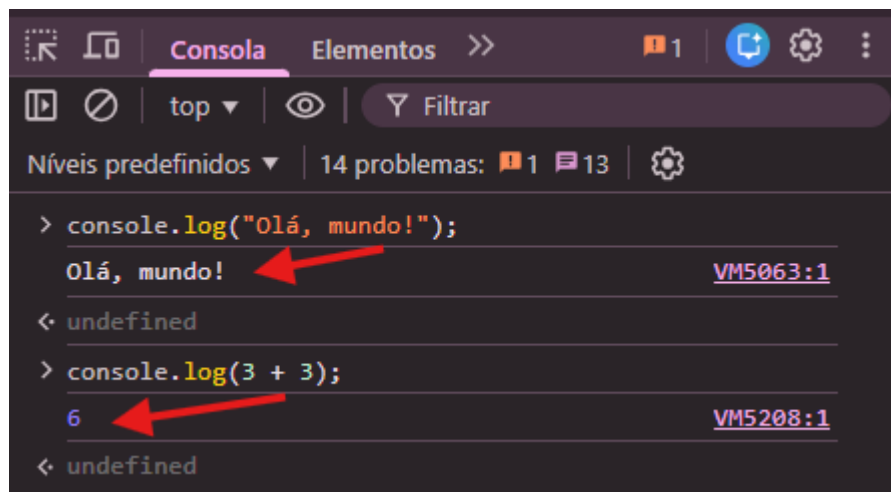
1. **console.log()**: É a ferramenta do desenvolvedor. Mostra os dados "por baixo do capô". Ótimo para testar cálculos.

```
console.log(<argumento>);
```

- Podemos utilizar o comando por completo também:

```
window.console.log(<argumento>);
```

<argumento> : quer dizer que podemos inserir valores de textos, números, expressões matemáticas, ou mesmo variáveis, entre outros tipos de dados. E ao executar o comando, o motor que vai interpretar o Javascript vai exibir o que estiver inserido nestes "argumentos" na tela que vimos o *item 1* deste resumo.



Tela do console do navegador exibindo mensagens ao executar comandos.

2. **alert()**: É a ferramenta do usuário. Uma janela que interrompe a navegação para dar um aviso.

```
alert(<argumento>);  
window.alert(<argumento>);
```

As regras para sobre os argumentos são iguais ao `console.log()`.

3. AS CAIXAS MÁGICAS (VARIÁVEIS)

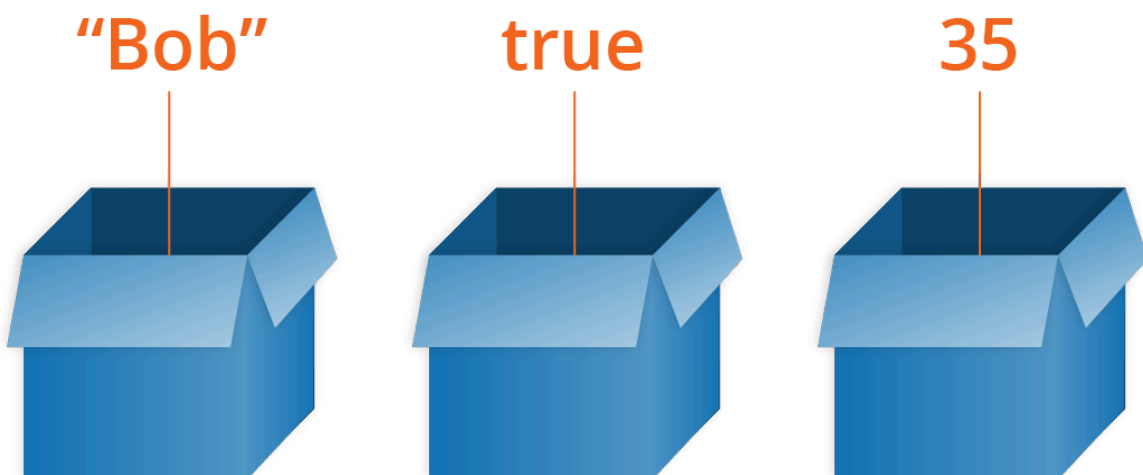
Uma variável é um espaço na memória do computador. Pense no trio:

- **Etiqueta (Nome):** Como chamamos o dado (ex: nomeUsuario).
 - Isto significa que **toda variável tem um nome para identificá-lo**.
- **Conteúdo (Valor):** O que está guardado dentro (ex: "João").
 - Podemos guardar dados dentro destas variáveis para recuperar depois.
- **Ação (Atribuição):** O símbolo = significa "recebe".
 - Ao atribuir um valor ele estará guardado dentro da variável.

Sintaxe:

```
let <nome_da_variavel> = <valor> ;
```

- **let:** comando Javascript para criar uma variável do tipo `let`;
- **<nome_da_variavel>:** local onde você vai inserir o identificador da sua variável recém criada;
- **=** → lê-se "recebe" - é o **sinal de atribuição**;
- **<valor>:** é o valor que você vai guardar dentro da variável que você acabou de criar e nomear.



Representação de variáveis guardando um texto, um booleano e um número.

Na imagem acima, temos uma ilustração de três variáveis sendo criadas (**ainda sem identificador - sem nome**) apenas que você compreenda que **variáveis nada mais são que reservar um espaço para guardar dados**.

Regras de Ouro:

Em Javascript, como vimos, temos basicamente três formas de se declarar uma variável e para isto precisamos usar umas das **palavras-chave (palavras reservadas)** abaixo.

- **let:** Para valores que podem mudar (ex: saldo da conta).
 - Não é obrigatório atribuir valor na criação. Se não fizer, ele guarda o tipo de dado **underfined** (valor não definido).
- **const:** Para valores que nunca mudam (ex: seu CPF).
 - Devem sempre ter valor atribuído na criação.
- **var: NÃO USAR.** É uma forma antiga e confusa de declarar variáveis.

As variáveis precisam ser identificadas, logo damos nomes a elas.

- **Nomes:**
 - Sem espaços, sem começar com números e use o padrão camelCase (ex: `minhaVariavelExemplo`);
 - Podem começar com underscore → (`_`) ou com cifrão → (`$`): (ex: `_visitante` - `$mandante`). Apesar de não ser comum declarações desta maneira;
 - O conselho é sempre nomear suas variáveis com nomes que façam sentido, **nunca assim:**

```
let x = 5;  
console.log(x) // → saída 5.
```

- Afinal de contas, o nome `x` não tem um significado que faça sentido para nós desenvolvedores e pode representar qualquer coisa deixando nosso código confuso.
- Javascript é uma linguagem de programação 'non-sensitive', isto quer dizer que **ele diferenciar letras maiúsculas de minúsculas**. Portanto uma variável chamada `pessoa` é diferente de `Pessoa`, que também é diferente de `PESSOA`. Cuidado ao nomear sua variável.

Para utilizá-la você precisa usar o nome exato com o qual foi criado. Na dúvida, copie o nome e cole para não haver erro de digitação.

 **Palavras reservadas** significa palavras que possuem algum valor semântico para a linguagem, isto é, representam alguma funcionalidade. “Lembra que conversamos que praticamente toda linha é um comando em Javascript? Pois bem, palavras como `let`, `var` e `const`, assim como `console` e `alert`, são palavras reservadas às suas utilizações, **não podendo ser utilizadas para nomear variáveis**”.

Variáveis podem receber valores de outras variáveis

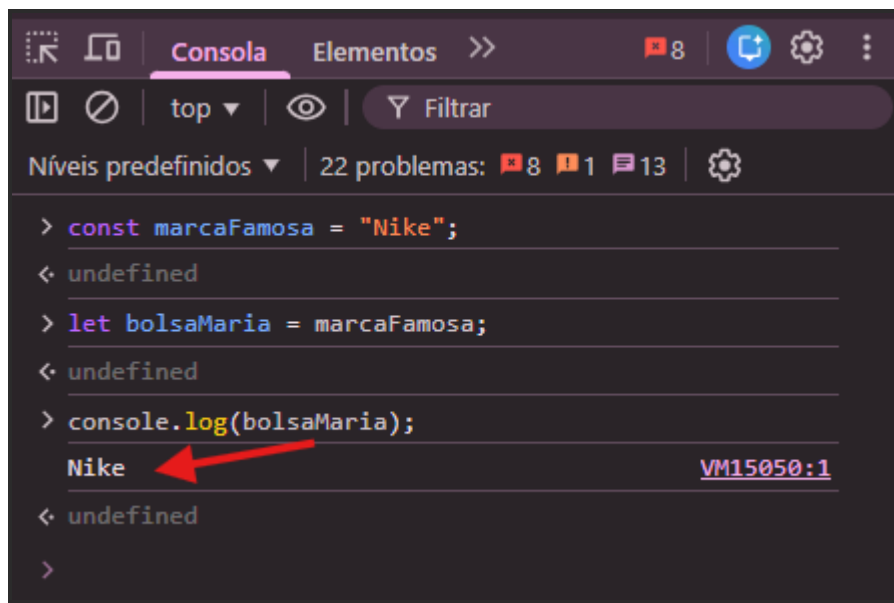
Assim como podemos atribuir valores para dentro de variáveis, também podemos atribuir outras variáveis para dentro de uma variável, fazendo com que ela receba o valor da variável que foi atribuída. “*Não fez sentido? Calma! Observe!*”

Suponha que você tenha uma variável chamada `marcaFamosa` e dentro dela tenha uma marca famosa de sapato. E você tem outra variável chamada `bolsaDaMaria`. Esta variável fictícia representa o que a dona Maria gostaria de levar para seu filho José. Assim, para “caixa” dela, vamos atribuir o valor que tem dentro da variável `marcaFamosa` para dentro da variável da dona Maria. Observe:

```
const marcaFamosa = "Nike";

let bolsaMaria = marcaFamosa;

console.log(bolsaMaria);
// Saída no console será → "Nike"
```



```
> const marcaFamosa = "Nike";
< undefined
> let bolsaMaria = marcaFamosa;
< undefined
> console.log(bolsaMaria);
Nike
< undefined
>
```

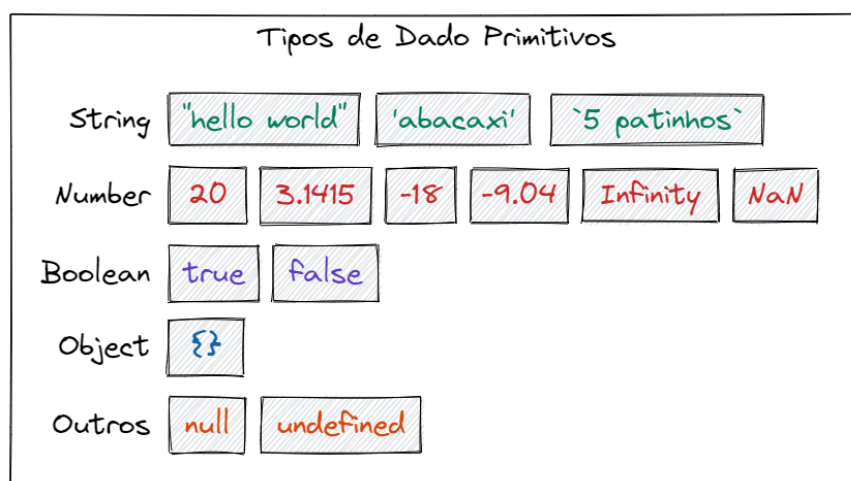
The screenshot shows a browser's developer console with the 'Console' tab selected. It displays a series of JavaScript commands and their results. The first command is `const marcaFamosa = "Nike";`, which returns `undefined`. The second is `let bolsaMaria = marcaFamosa;`, also returning `undefined`. The third is `console.log(bolsaMaria);`, which outputs the string `Nike`. A red arrow points to the word `Nike` in the output. The console interface includes a search bar, a filter button, and a status bar indicating 22 problems (8 errors, 1 warning, 13 info).

4. TIPOS DE DADOS E TIPAGEM FRACA

O JavaScript é esperto (e às vezes perigoso). Ele adivinha o tipo do dado pelo valor que você coloca:

- **String:** Texto. Sempre entre aspas (ex: "10" é um texto).
- **Number:** Número puro. Usado para cálculos (ex: 10 é um número).
- **Tipagem Fraca:** Você pode criar uma variável com um número e depois colocar um texto nela. *Cuidado: com grandes poderes vêm grandes responsabilidades.*

"Ainda vamos conversar sobre outros tipos de dados depois."



⚙️ 5. O MÉTODO IPO (ENTRADA -> PROCESSAMENTO -> SAÍDA)

O foco desta UC não é decorar comandos, mas **resolver problemas**.

1. **Input (Entrada):** Quais dados eu preciso para resolver isso?
 2. **Process (Processamento):** Qual é o passo a passo lógico (Algoritmo)?
 3. **Output (Saída):** O que eu preciso mostrar para o usuário final?
-

🔥 DESAFIO DE FIXAÇÃO

Baseado no que você aprendeu até agora, escreva linha a linha, no papel mesmo, qual valor você acha que vai ser exibido no console do desenvolvedor.

JavaScript

```
1  let garrafas = 5;  
2  
3  let preco = 2;  
4  
5  let total = garrafas * preco;  
6  
7  garrafas = 10;  
8  
9  total = garrafas * preco;  
10  
11 console.log(total); // Qual valor aparecerá aqui?
```

DICA: O símbolo de asterístico significa multiplicação em operações matemáticas.



NOTA DO INSTRUTOR:

*"Não se assustem com o código agora. O código é apenas a ferramenta. O que eu quero de vocês é o **raciocínio**. Se você consegue quebrar um problema em passos, você já é um programador. A sintaxe a gente aprende com o tempo e com a raiva de não ter o VS Code ainda!"*